

OSSP-PRACTICAL-WEEK7

TASK1:

Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Source code:

```
GNU nano 7.2                                memory.c *
#include <stdio.h>
#include <stdlib.h>

int global_var = 10;

void code_function()
{
    printf("Code address    : %p\n", (void *)code_function);
}

int main()
{
    static int static_var = 20;
    int stack_var = 30;

    int *heap_var = malloc(sizeof(int));
    *heap_var = 40;

    printf("Linux Process Address Space\n");
    printf("-----\n");

    code_function();

    printf("Global address : %p\n", (void *)&global_var);
    printf("Static address : %p\n", (void *)&static_var);
    printf("Heap address   : %p\n", (void *)heap_var);
    printf("Stack address  : %p\n", (void *)&stack_var);

    free(heap_var);

    return 0;
}
```

Output:

```
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806$ ./memory
Linux Process Address Space
-----
Code address   : 0x5704e903f1c9
Global address : 0x5704e9042010
Static address : 0x5704e9042014
Heap address   : 0x570508b182a0
Stack address  : 0x7fffe8a2d23c
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806$ |
```

Observation:

I learned how a Linux process is organized into different memory segments such as code, global, static, heap, and stack. I learned how to print the memory addresses of variables belonging to these segments and understood that dynamically allocated memory is stored in the heap while local variables are stored in the stack.

Task2:

Make use of the `/proc/[pid]/maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

Source code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 100;

int main()
{
    static int static_var = 200;
    int stack_var = 300;

    int *heap_var = malloc(sizeof(int));
    *heap_var = 400;

    printf("Process Memory Analysis\n");
    printf("-----\n");
    printf("PID          : %d\n", getpid());
    printf("Global address: %p\n", (void *)&global_var);
    printf("Static address: %p\n", (void *)&static_var);
    printf("Heap address  : %p\n", (void *)heap_var);
    printf("Stack address : %p\n", (void *)&stack_var);

    printf("\nProcess is running...\n");
    printf("Press Ctrl+C to terminate.\n");

    while (1)
    {
        sleep(5);
    }

    free(heap_var);
    return 0;
}
```

Output:

```
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806$ cat /proc/552/maps | grep -E '\[heap\]|\[stack\]|vmemory'
600939c68000-600939c69000 r--p 00000000 00:46 166070236259296741 /mnt/c/Users/SAIRAM0806/vmemory
600939c69000-600939c6a000 r-xp 00001000 00:46 166070236259296741 /mnt/c/Users/SAIRAM0806/vmemory
600939c6a000-600939c6b000 r--p 00002000 00:46 166070236259296741 /mnt/c/Users/SAIRAM0806/vmemory
600939c6b000-600939c6c000 r--p 00002000 00:46 166070236259296741 /mnt/c/Users/SAIRAM0806/vmemory
600939c6c000-600939c6d000 rw-p 00003000 00:46 166070236259296741 /mnt/c/Users/SAIRAM0806/vmemory
6009430dc000-6009430fd000 rw-p 00000000 00:00 0 [heap]
7ffc7c389000-7ffc7c3aa000 rw-p 00000000 00:00 0 [stack]
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806$ --|
```

```
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806$ cat /proc/552/status | grep -E "VmSize|VmRSS|VmData|VmStk"
VmSize: 2692 kB
VmRSS: 1736 kB
VmData: 224 kB
VmStk: 132 kB
sairam0806@LAPTOP-M6NQIUAP:/mnt/c/Users/SAIRAM0806$ |
```

Observation:

I learned how to examine the virtual memory layout of a running Linux process using `/proc/[PID]/maps`. I understood how memory mappings represent code, data, heap, stack, and shared libraries, along with their read, write, and execute permissions. I also learned how tools such as `pmap` can be used to analyze process memory usage.